

PROTOTYPE PATTERN

SOLUTION

Prototype Class

```
public abstract class Vehicle {
    private String brand;
    private String model;
    private String color;

    protected Vehicle(Vehicle vehicle) {
        this.brand = vehicle.brand;
        this.model = vehicle.model;
        this.color = vehicle.color;
    }

    public abstract Vehicle clone();
}
```

```
public class Car extends Vehicle {
    private int topSpeed;

    public Car() { }
    public Car(Car car) {
        super(car);
        this.topSpeed = car.topSpeed;
    }

    @Override
    public Car clone() {
        return new Car(this);
    }
}
```

```
public void clone(List<Vehicle> vehicles) {
    List<Vehicle> copyList = new ArrayList<>();
    for (Vehicle vehicle : vehicles) {
        copyList.add(vehicle.clone());
    }
} // client using the clone method.
```

```
public class Bus extends Vehicle {
    private int doors;
    ...

    @Override
    public Bus clone() {
        return new Bus(this);
    }
}
```

```
public void clone(List<Vehicle> vehicles) {
    List<Vehicle> copyList = new ArrayList<>();
    for (Vehicle vehicle : vehicles) {
        if (vehicle instanceof Car) {
            // Add copy of Car object to List
        } else if (vehicle instanceof Bus) {
            // Add copy of Bus object to List
        }
    }
}
```

→ Every clone operation will require object type checks.

we are coupled with the concrete classes if we don't make use of the prototype pattern

Note:
The prototype pattern has a lot of similarities with the Abstract factory pattern.

IMP!!

Shallow vs Deep Copy:-

(Suppose the object contains a GPS system)

```
public class Car extends Vehicle {
    private int topSpeed;
    private GpsSystem gpsSystem;

    public Car() { }

    public Car(Car car) {
        super(car);
        this.topSpeed = car.topSpeed;
        this.gpsSystem = new GpsSystem();
        // or gpsSystem.clone();
    }

    @Override
    public Car clone() {
        return new Car(this);
    }
}
```

→ If we do a shallow copy then two car objects A & B gets a pointer to the same GPS system object i.e chaotic

→ Deep copy will be the correct approach so that every new car object creates a copy of its own GPS system i.e the GPS system should also follow the prototype pattern.

The Prototype pattern is a creational design pattern used when creating a copy of object from scratch is complex, slow or expensive.

KEY COMPONENTS

1. Prototype: declares an interface for cloning itself.
2. Concrete Prototype: Implements the interface with the logic to copy its own state and data.
3. Client: Requests a new object by asking prototype to clone itself.

INTUITION

→ Suppose you have a "Car" object and you are asked to create 10 copies of the Car → Now naive approach will be to run a loop and create new object at each iteration and copy the member values of original object → The issue will be: Some of the variables can be private i.e not accessible / Also it defies SOLID principles

Potential Issue:

CAR A = new CAR();
for (.....) {
 CAR i = new CAR();
 i.color = A.color;
 i.number = A.number;
}

→ Suppose we have a general interface like "Vehicle" instead of Car or BUS then?

This can be a private member of the class

Issue 2: Tight coupling bcz you need to know the exact member variables present in the class to create a copy i.e the color or number.

Issue 1